Complete PostgreSQL Mastery Guide

Table of Contents

Part 1: Foundation

Part 2: Core Operations

Part 3: Advanced Querying

Part 4: Advanced Concepts

---

1. PostgreSQL Basics & Setup

Installation & Connection

```
# Install PostgreSQL (Ubuntu/Debian)
sudo apt update
sudo apt install postgresql postgresql-contrib

# Start PostgreSQL service
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Connect to PostgreSQL
sudo -u postgres psql

# Connect with specific user and database
psql -h localhost -U username -d database_name
```

Basic Commands

```
-- List all databases
\l

-- Connect to a database
\c database_name

-- List all tables
\dt

-- Describe table structure
\d table_name

-- Exit psql
\q
```

---

2. Database & Table Management

Database Operations

```
-- Create database
CREATE DATABASE company_db;

-- Drop database
DROP DATABASE IF EXISTS old_db;

-- Create schema
CREATE SCHEMA hr;
```

Table Creation & Management

```
-- Create table with constraints
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    hire_date DATE DEFAULT CURRENT_DATE,
    salary DECIMAL(10,2) CHECK (salary > 0),
    department_id INTEGER REFERENCES departments(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Add column
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);

-- Modify column
ALTER TABLE employees ALTER COLUMN phone SET NOT NULL;

-- Drop column
ALTER TABLE employees DROP COLUMN phone;

-- Add constraint
ALTER TABLE employees ADD CONSTRAINT salary_check CHECK (salary BETWEEN 1000 AND 1000000);

-- Create index
CREATE INDEX idx_employee_email ON employees(email);
CREATE INDEX idx_employee_name ON employees(first_name, last_name);
```

---

3. Data Types

Common Data Types

```
-- Numeric types
SMALLINT        -- 2 bytes, -32,768 to 32,767
INTEGER         -- 4 bytes, -2,147,483,648 to 2,147,483,647
BIGINT          -- 8 bytes, large range
SERIAL          -- Auto-incrementing integer
DECIMAL(p,s)    -- Exact decimal
NUMERIC(p,s)    -- Same as DECIMAL
REAL            -- 4-byte floating point
DOUBLE PRECISION -- 8-byte floating point

-- Character types
CHAR(n)         -- Fixed-length string
VARCHAR(n)      -- Variable-length string with limit
TEXT            -- Variable-length string without limit

-- Date/Time types
DATE            -- Date only (YYYY-MM-DD)
TIME            -- Time only (HH:MM:SS)
TIMESTAMP       -- Date and time
TIMESTAMPTZ     -- Date and time with timezone
INTERVAL        -- Time interval

-- Boolean
BOOLEAN         -- TRUE, FALSE, or NULL

-- JSON types
JSON            -- JSON data (stored as text)
JSONB           -- Binary JSON (faster operations)

-- Arrays
INTEGER[]       -- Array of integers
TEXT[]          -- Array of text

-- Example usage
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10,2),
    tags TEXT[],
    metadata JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

---

4. CRUD Operations

CREATE (INSERT)

```sql
-- Insert single record
INSERT INTO employees (first_name, last_name, email, salary, department_id)
VALUES ('John', 'Doe', 'john.doe@company.com', 75000.00, 1);

-- Insert multiple records
INSERT INTO employees (first_name, last_name, email, salary, department_id) VALUES
    ('Jane', 'Smith', 'jane.smith@company.com', 80000.00, 2),
    ('Bob', 'Johnson', 'bob.johnson@company.com', 65000.00, 1),
    ('Alice', 'Williams', 'alice.williams@company.com', 90000.00, 3);

-- Insert with returning
INSERT INTO employees (first_name, last_name, email, salary, department_id)
VALUES ('Mike', 'Davis', 'mike.davis@company.com', 70000.00, 2)
RETURNING id, created_at;

-- Insert from SELECT
INSERT INTO archived_employees
SELECT * FROM employees WHERE hire_date < '2020-01-01';

-- Insert with ON CONFLICT (UPSERT)
INSERT INTO employees (email, first_name, last_name, salary)
VALUES ('john.doe@company.com', 'John', 'Doe', 76000.00)
ON CONFLICT (email)
DO UPDATE SET
    salary = EXCLUDED.salary,
    updated_at = CURRENT_TIMESTAMP;
```

READ (SELECT)

```sql
-- Basic SELECT
SELECT * FROM employees;

-- Select specific columns
SELECT first_name, last_name, email FROM employees;

-- Select with alias
SELECT
    first_name AS "First Name",
    last_name AS "Last Name",
    salary * 12 AS "Annual Salary"
FROM employees;

-- Select with WHERE clause
SELECT * FROM employees WHERE salary > 70000;
SELECT * FROM employees WHERE department_id = 1 AND salary BETWEEN 60000 AND 80000;
SELECT * FROM employees WHERE first_name LIKE 'J%';
SELECT * FROM employees WHERE email ILIKE '%gmail.com';

-- Select with NULL handling
SELECT * FROM employees WHERE phone IS NULL;
SELECT * FROM employees WHERE phone IS NOT NULL;
```

UPDATE

```
-- Update single record
UPDATE employees
SET salary = 82000.00, updated_at = CURRENT_TIMESTAMP
WHERE id = 1;

-- Update multiple records
UPDATE employees
SET salary = salary * 1.1
WHERE department_id = 2;

-- Update with subquery
UPDATE employees
SET salary = (
    SELECT AVG(salary) * 1.2
    FROM employees e2
    WHERE e2.department_id = employees.department_id
)
WHERE performance_rating = 'Excellent';

-- Update with RETURNING
UPDATE employees
SET salary = salary * 1.05
WHERE department_id = 1
RETURNING id, first_name, last_name, salary;
```

DELETE

```
-- Delete specific records
DELETE FROM employees WHERE id = 5;

-- Delete with conditions
DELETE FROM employees WHERE hire_date < '2019-01-01';

-- Delete with subquery
DELETE FROM employees
WHERE id IN (
    SELECT e.id
    FROM employees e
    JOIN departments d ON e.department_id = d.id
    WHERE d.name = 'Discontinued Department'
);

-- Delete with RETURNING
DELETE FROM employees
WHERE salary < 50000
RETURNING id, first_name, last_name;

-- Truncate table (faster for deleting all records)
TRUNCATE TABLE temp_employees;
```

---

5. Query Fundamentals

WHERE Clause Operators

```sql
-- Comparison operators
SELECT * FROM employees WHERE salary = 75000;
SELECT * FROM employees WHERE salary <> 75000;  -- Not equal
SELECT * FROM employees WHERE salary != 75000;  -- Not equal (alternative)
SELECT * FROM employees WHERE salary > 70000;
SELECT * FROM employees WHERE salary >= 70000;
SELECT * FROM employees WHERE salary < 80000;
SELECT * FROM employees WHERE salary <= 80000;

-- Logical operators
SELECT * FROM employees WHERE salary > 70000 AND department_id = 1;
SELECT * FROM employees WHERE salary > 90000 OR department_id = 3;
SELECT * FROM employees WHERE NOT (salary < 60000);

-- Pattern matching
SELECT * FROM employees WHERE first_name LIKE 'J%';        -- Starts with J
SELECT * FROM employees WHERE first_name LIKE '%n';        -- Ends with n
SELECT * FROM employees WHERE first_name LIKE '%oh%';      -- Contains oh
SELECT * FROM employees WHERE first_name ILIKE 'JOHN';     -- Case insensitive
SELECT * FROM employees WHERE first_name ~ '^[A-M]';       -- Regex: starts with A-M

-- Range operators
SELECT * FROM employees WHERE salary BETWEEN 60000 AND 80000;
SELECT * FROM employees WHERE hire_date BETWEEN '2020-01-01' AND '2023-12-31';

-- List operators
SELECT * FROM employees WHERE department_id IN (1, 2, 3);
SELECT * FROM employees WHERE first_name IN ('John', 'Jane', 'Bob');

-- NULL checks
SELECT * FROM employees WHERE phone IS NULL;
SELECT * FROM employees WHERE phone IS NOT NULL;
```

DISTINCT and CASE

```sql
-- DISTINCT values
SELECT DISTINCT department_id FROM employees;
SELECT DISTINCT department_id, job_title FROM employees;

-- CASE statements
SELECT
    first_name,
    last_name,
    salary,
    CASE
        WHEN salary >= 90000 THEN 'High'
        WHEN salary >= 70000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_category
FROM employees;

-- Simple CASE
SELECT
    first_name,
    CASE department_id
        WHEN 1 THEN 'Engineering'
        WHEN 2 THEN 'Marketing'
        WHEN 3 THEN 'Sales'
        ELSE 'Other'
    END AS department_name
FROM employees;
```

---

6. Joins

Inner Join

```sql
-- Basic INNER JOIN
SELECT
    e.first_name,
    e.last_name,
    d.name AS department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;

-- Multiple table INNER JOIN
SELECT
    e.first_name,
    e.last_name,
    d.name AS department_name,
    p.title AS project_title
FROM employees e
INNER JOIN departments d ON e.department_id = d.id
INNER JOIN employee_projects ep ON e.id = ep.employee_id
INNER JOIN projects p ON ep.project_id = p.id;
```

Left Join

```sql
-- LEFT JOIN (includes all employees, even without departments)
SELECT
    e.first_name,
    e.last_name,
    COALESCE(d.name, 'No Department') AS department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id;

-- Finding records without matches
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
WHERE d.id IS NULL;
```

Right Join

```sql
-- RIGHT JOIN (includes all departments, even without employees)
SELECT
    d.name AS department_name,
    e.first_name,
    e.last_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.id;
```

Full Outer Join

```sql
-- FULL OUTER JOIN (includes all records from both tables)
SELECT
    e.first_name,
    e.last_name,
    d.name AS department_name
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.id;
```

Cross Join

```sql
-- CROSS JOIN (Cartesian product)
SELECT
    e.first_name,
    p.title
FROM employees e
CROSS JOIN projects p;
```

Self Join

```sql
-- Self JOIN (finding employees and their managers)
SELECT
    emp.first_name AS employee_name,
    mgr.first_name AS manager_name
FROM employees emp
LEFT JOIN employees mgr ON emp.manager_id = mgr.id;
```

Join Conditions and Performance

```
-- Multiple join conditions
SELECT *
FROM employees e
JOIN departments d ON e.department_id = d.id AND d.active = true;

-- Using USING clause (when column names are the same)
SELECT *
FROM employees e
JOIN departments d USING (department_id);

-- Join with additional filtering
SELECT
    e.first_name,
    e.last_name,
    d.name
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.salary > 70000 AND d.budget > 100000;
```

---

7. Sorting & Filtering

ORDER BY

```sql
-- Basic sorting
SELECT * FROM employees ORDER BY salary;                    -- Ascending (default)
SELECT * FROM employees ORDER BY salary DESC;               -- Descending
SELECT * FROM employees ORDER BY salary ASC;                -- Explicit ascending

-- Multiple column sorting
SELECT * FROM employees
ORDER BY department_id ASC, salary DESC;

-- Sorting by expression
SELECT
    first_name,
    last_name,
    salary,
    salary * 12 AS annual_salary
FROM employees
ORDER BY salary * 12 DESC;

-- Sorting with NULL handling
SELECT * FROM employees ORDER BY phone NULLS FIRST;         -- NULLs at beginning
SELECT * FROM employees ORDER BY phone NULLS LAST;          -- NULLs at end

-- Sorting by column position
SELECT first_name, last_name, salary
FROM employees
ORDER BY 3 DESC, 1 ASC;  -- Order by 3rd column (salary) desc, then 1st column (first_name) asc

-- Custom sorting with CASE
SELECT *
FROM employees
ORDER BY
    CASE department_id
        WHEN 1 THEN 1  -- Engineering first
        WHEN 3 THEN 2  -- Sales second
        WHEN 2 THEN 3  -- Marketing third
        ELSE 4
    END,
    salary DESC;
```

Advanced Filtering

```sql
-- Date filtering
SELECT * FROM employees WHERE hire_date >= '2022-01-01';
SELECT * FROM employees WHERE EXTRACT(YEAR FROM hire_date) = 2022;
SELECT * FROM employees WHERE hire_date >= CURRENT_DATE - INTERVAL '1 year';

-- String functions in WHERE
SELECT * FROM employees WHERE LENGTH(first_name) > 5;
SELECT * FROM employees WHERE UPPER(first_name) = 'JOHN';
SELECT * FROM employees WHERE first_name SIMILAR TO '(John|Jane)';

-- Numeric filtering
SELECT * FROM employees WHERE MOD(salary, 1000) = 0;  -- Salary divisible by 1000
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
```

8. Grouping & Aggregation

GROUP BY Basics

```
-- Basic grouping
SELECT department_id, COUNT(*) as employee_count
FROM employees
GROUP BY department_id;

-- Multiple grouping columns
SELECT
    department_id,
    EXTRACT(YEAR FROM hire_date) as hire_year,
    COUNT(*) as employee_count
FROM employees
GROUP BY department_id, EXTRACT(YEAR FROM hire_date)
ORDER BY department_id, hire_year;

-- Grouping with joins
SELECT
    d.name as department_name,
    COUNT(e.id) as employee_count,
    AVG(e.salary) as avg_salary
FROM departments d
LEFT JOIN employees e ON d.id = e.department_id
GROUP BY d.id, d.name;
```

Aggregate Functions

```
-- Common aggregate functions
SELECT
    COUNT(*) as total_employees,
    COUNT(phone) as employees_with_phone,  -- COUNT ignores NULLs
    SUM(salary) as total_salary_cost,
    AVG(salary) as average_salary,
    MIN(salary) as min_salary,
    MAX(salary) as max_salary,
    STDDEV(salary) as salary_std_deviation
FROM employees;

-- String aggregation
SELECT
    department_id,
    STRING_AGG(first_name || ' ' || last_name, ', ' ORDER BY last_name) as employee_names
FROM employees
GROUP BY department_id;

-- Array aggregation
SELECT
    department_id,
    ARRAY_AGG(salary ORDER BY salary DESC) as all_salaries
FROM employees
GROUP BY department_id;
```

HAVING Clause

```
-- HAVING for filtering groups
SELECT
    department_id,
    COUNT(*) as employee_count,
    AVG(salary) as avg_salary
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;  -- Only departments with more than 5 employees

-- HAVING with multiple conditions
SELECT
    department_id,
    COUNT(*) as employee_count,
    AVG(salary) as avg_salary
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 3 AND AVG(salary) > 70000;

-- HAVING vs WHERE
SELECT
    department_id,
    COUNT(*) as senior_employee_count
FROM employees
WHERE salary > 60000  -- Filter rows before grouping
GROUP BY department_id
HAVING COUNT(*) > 2;  -- Filter groups after grouping
```

GROUPING SETS, ROLLUP, and CUBE

```
-- GROUPING SETS (custom grouping combinations)
SELECT
    department_id,
    EXTRACT(YEAR FROM hire_date) as hire_year,
    COUNT(*) as employee_count
FROM employees
GROUP BY GROUPING SETS (
    (department_id),
    (EXTRACT(YEAR FROM hire_date)),
    (department_id, EXTRACT(YEAR FROM hire_date)),
    ()  -- Grand total
);

-- ROLLUP (hierarchical grouping)
SELECT
    department_id,
    job_title,
    COUNT(*) as employee_count
FROM employees
GROUP BY ROLLUP (department_id, job_title);

-- CUBE (all possible combinations)
SELECT
    department_id,
    job_title,
    COUNT(*) as employee_count
FROM employees
GROUP BY CUBE (department_id, job_title);
```

---

9. Pagination

LIMIT and OFFSET

```
-- Basic pagination
SELECT * FROM employees
ORDER BY id
LIMIT 10;  -- First 10 records

-- Skip records with OFFSET
SELECT * FROM employees
ORDER BY id
LIMIT 10 OFFSET 20;  -- Records 21-30

-- Alternative syntax
SELECT * FROM employees
ORDER BY id
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

Cursor-based Pagination

```
-- More efficient for large datasets
-- Page 1
SELECT * FROM employees
WHERE id > 0
ORDER BY id
LIMIT 10;

-- Next page (assuming last id from previous page was 10)
SELECT * FROM employees
WHERE id > 10
ORDER BY id
LIMIT 10;

-- With timestamp cursor
SELECT * FROM employees
WHERE created_at > '2023-01-01 10:30:00'
ORDER BY created_at, id
LIMIT 10;
```

Pagination with Total Count

```
-- Get page data and total count in one query
SELECT
    *,
    COUNT(*) OVER() as total_count
FROM employees
ORDER BY id
LIMIT 10 OFFSET 20;

-- Or use a window function
WITH paginated_data AS (
    SELECT
        *,
        ROW_NUMBER() OVER (ORDER BY id) as row_num
    FROM employees
)
SELECT * FROM paginated_data
WHERE row_num BETWEEN 21 AND 30;
```

---

10. JSON Operations

JSON vs JSONB

```
-- Create table with JSON columns
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    metadata JSON,      -- Stores exact JSON text
    attributes JSONB    -- Stores binary representation (faster operations)
);

-- Insert JSON data
INSERT INTO products (name, metadata, attributes) VALUES
('Laptop',
 '{"brand": "Dell", "model": "XPS 13", "specs": {"ram": "16GB", "storage": "512GB SSD"}}',
 '{"color": "silver", "weight": "1.2kg", "features": ["touchscreen", "backlit keyboard"]}'
);
```

JSON Operators

```
-- Extract JSON field (returns JSON)
SELECT metadata -> 'brand' FROM products;          -- Returns: "Dell"
SELECT metadata ->> 'brand' FROM products;         -- Returns: Dell (as text)

-- Extract nested JSON
SELECT metadata -> 'specs' -> 'ram' FROM products; -- Returns: "16GB"
SELECT metadata #> '{specs,ram}' FROM products;    -- Same as above
SELECT metadata #>> '{specs,ram}' FROM products;   -- Returns: 16GB (as text)

-- Check if key exists
SELECT * FROM products WHERE metadata ? 'brand';
SELECT * FROM products WHERE attributes ?& array['color', 'weight'];  -- All keys exist
SELECT * FROM products WHERE attributes ?| array['color', 'size'];    -- Any key exists

-- JSONB containment
SELECT * FROM products WHERE attributes @> '{"color": "silver"}';
SELECT * FROM products WHERE '{"color": "silver", "weight": "1.2kg"}' @> attributes;
```

JSON Functions

```
-- json_agg: Aggregate rows into JSON array
SELECT
    d.name as department,
    json_agg(
        json_build_object(
            'name', e.first_name || ' ' || e.last_name,
            'salary', e.salary,
            'email', e.email
        ) ORDER BY e.salary DESC
    ) as employees
FROM departments d
LEFT JOIN employees e ON d.id = e.department_id
GROUP BY d.id, d.name;

-- json_build_object: Build JSON object from key-value pairs
SELECT
    json_build_object(
        'employee_id', id,
        'full_name', first_name || ' ' || last_name,
        'contact', json_build_object(
            'email', email,
            'phone', phone
        ),
        'employment', json_build_object(
            'salary', salary,
            'hire_date', hire_date,
            'department_id', department_id
        )
    ) as employee_json
FROM employees;

-- json_build_array: Build JSON array
SELECT
    json_build_array(first_name, last_name, salary, department_id) as employee_array
FROM employees;

-- json_object_agg: Create JSON object from key-value pairs
SELECT
    json_object_agg(first_name || ' ' || last_name, salary) as salary_lookup
FROM employees;

-- json_array_elements: Expand JSON array to rows
SELECT
    id,
    json_array_elements(attributes -> 'features') as feature
FROM products
WHERE attributes -> 'features' IS NOT NULL;

-- json_each: Expand JSON object to key-value pairs
SELECT
    id,
    key,
    value
FROM products,
     json_each(metadata -> 'specs');
```

JSON Modification (JSONB only)

```
-- Update JSONB field
UPDATE products
SET attributes = attributes || '{"price": 999.99}'::jsonb
WHERE id = 1;

-- Remove key from JSONB
UPDATE products
SET attributes = attributes - 'weight'
WHERE id = 1;

-- Update nested JSONB value
UPDATE products
SET metadata = jsonb_set(metadata, '{specs,ram}', '"32GB"', false)
WHERE id = 1;

-- Add to JSONB array
UPDATE products
SET attributes = jsonb_set(
    attributes,
    '{features}',
    (attributes -> 'features') || '["wireless charging"]'::jsonb
)
WHERE id = 1;
```

JSON Indexing

```
-- Index on JSONB field
CREATE INDEX idx_attributes_color ON products USING gin ((attributes -> 'color'));

-- GIN index for full JSONB document
CREATE INDEX idx_attributes_gin ON products USING gin (attributes);

-- Index on extracted JSON value
CREATE INDEX idx_metadata_brand ON products ((metadata ->> 'brand'));
```

Advanced JSON Queries

```
-- JSON path queries (PostgreSQL 12+)
SELECT * FROM products
WHERE attributes @? '$.features[*] ? (@ == "touchscreen")';

-- Complex JSON aggregation with filtering
SELECT
    EXTRACT(YEAR FROM created_at) as year,
    json_build_object(
        'total_products', COUNT(*),
        'by_color', json_object_agg(
            attributes ->> 'color',
            COUNT(*)
        ) FILTER (WHERE attributes ? 'color'),
        'average_price', AVG((attributes ->> 'price')::numeric),
        'features_summary', (
            SELECT json_object_agg(feature, feature_count)
            FROM (
                SELECT
                    json_array_elements_text(attributes -> 'features') as feature,
                    COUNT(*) as feature_count
                FROM products p2
                WHERE EXTRACT(YEAR FROM p2.created_at) = EXTRACT(YEAR FROM products.created_at)
                  AND attributes -> 'features' IS NOT NULL
                GROUP BY feature
            ) feature_stats
        )
    ) as year_summary
FROM products
GROUP BY EXTRACT(YEAR FROM created_at)
ORDER BY year;
```

---

11. Subqueries & CTEs

Subqueries

```sql
-- Scalar subquery (returns single value)
SELECT
    first_name,
    last_name,
    salary,
    (SELECT AVG(salary) FROM employees) as avg_company_salary,
    salary - (SELECT AVG(salary) FROM employees) as salary_diff
FROM employees;

-- Subquery in WHERE clause
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Subquery with IN
SELECT * FROM employees
WHERE department_id IN (
    SELECT id FROM departments WHERE budget > 500000
);

-- Correlated subquery
SELECT
    e1.first_name,
    e1.last_name,
    e1.salary,
    (SELECT COUNT(*)
     FROM employees e2
     WHERE e2.department_id = e1.department_id
       AND e2.salary > e1.salary
    ) as employees_earning_more
FROM employees e1;

-- EXISTS subquery
SELECT * FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e
    WHERE e.department_id = d.id AND e.salary > 80000
);

-- NOT EXISTS subquery
SELECT * FROM departments d
WHERE NOT EXISTS (
    SELECT 1 FROM employees e
    WHERE e.department_id = d.id
);
```

Common Table Expressions (CTEs)

```sql
-- Basic CTE
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 80000
)
SELECT
    d.name as department,
    COUNT(he.id) as high_earner_count
FROM departments d
LEFT JOIN high_earners he ON d.id = he.department_id
GROUP BY d.id, d.name;

-- Multiple CTEs
WITH
department_stats AS (
    SELECT
        department_id,
        COUNT(*) as emp_count,
        AVG(salary) as avg_salary,
        MAX(salary) as max_salary
    FROM employees
    GROUP BY department_id
),
high_budget_depts AS (
    SELECT id FROM departments WHERE budget > 1000000
)
SELECT
    d.name,
    ds.emp_count,
    ds.avg_salary,
    ds.max_salary,
    CASE WHEN hbd.id IS NOT NULL THEN 'High Budget' ELSE 'Normal Budget' END as budget_category
FROM departments d
JOIN department_stats ds ON d.id = ds.department_id
LEFT JOIN high_budget_depts hbd ON d.id = hbd.id;

-- Recursive CTE (for hierarchical data)
WITH RECURSIVE employee_hierarchy AS (
    -- Base case: top-level managers
    SELECT
        id,
        first_name,
        last_name,
        manager_id,
        0 as level,
        ARRAY[id] as path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case: employees with managers
    SELECT
        e.id,
        e.first_name,
        e.last_name,
        e.manager_id,
        eh.level + 1,
```

```
        eh.path || e.id
    FROM employees e
    JOIN employee_hierarchy eh ON e.manager_id = eh.id
    WHERE NOT e.id = ANY(eh.path)  -- Prevent infinite loops
)
SELECT
    REPEAT('  ', level) || first_name || ' ' || last_name as hierarchy,
    level,
    path
FROM employee_hierarchy
ORDER BY path;

-- CTE for data modification
WITH updated_salaries AS (
    UPDATE employees
    SET salary = salary * 1.1
    WHERE department_id = 1
    RETURNING id, first_name, last_name, salary
)
SELECT
    'Updated salary for ' || first_name || ' ' || last_name ||
    ' to $' || salary as update_message
FROM updated_salaries;
```

Window Functions in CTEs

```
-- Ranking employees within departments
WITH ranked_employees AS (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) as salary_rank,
        DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) as salary_dense_rank,
        PERCENT_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) as salary_percentile
    FROM employees
)
SELECT
    first_name,
    last_name,
    salary,
    salary_rank,
    CASE
        WHEN salary_rank = 1 THEN 'Top earner'
        WHEN salary_rank <= 3 THEN 'Top 3'
        ELSE 'Other'
    END as category
FROM ranked_employees
WHERE salary_rank <= 5;
```

---

12. Window Functions

Basic Window Functions

```sql
-- ROW_NUMBER: Unique sequential number
SELECT
    first_name,
    last_name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) as salary_rank
FROM employees;

-- RANK: Rank with gaps for ties
SELECT
    first_name,
    last_name,
    salary,
    RANK() OVER (ORDER BY salary DESC) as rank,
    DENSE_RANK() OVER (ORDER BY salary DESC) as dense_rank
FROM employees;

-- NTILE: Distribute rows into buckets
SELECT
    first_name,
    last_name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) as quartile
FROM employees;
```

Window Functions with PARTITION BY

```sql
-- Partition by department
SELECT
    first_name,
    last_name,
    department_id,
    salary,
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) as dept_salary_rank,
    COUNT(*) OVER (PARTITION BY department_id) as dept_employee_count,
    AVG(salary) OVER (PARTITION BY department_id) as dept_avg_salary
FROM employees;

-- Multiple window functions
SELECT
    first_name,
    last_name,
    department_id,
    salary,
    hire_date,
    -- Salary rankings
    RANK() OVER (ORDER BY salary DESC) as overall_salary_rank,
    RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) as dept_salary_rank,
    -- Running totals
    SUM(salary) OVER (ORDER BY hire_date) as running_salary_total,
    -- Comparisons
    salary - LAG(salary) OVER (ORDER BY salary DESC) as salary_gap_from_higher,
    LEAD(salary) OVER (ORDER BY salary DESC) - salary as salary_gap_to_lower
FROM employees;
```

Aggregate Window Functions

```
-- Running totals and averages
SELECT
    first_name,
    last_name,
    hire_date,
    salary,
    SUM(salary) OVER (ORDER BY hire_date) as running_total,
    AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) as running_avg,
    COUNT(*) OVER (ORDER BY hire_date) as running_count
FROM employees
ORDER BY hire_date;

-- Moving averages
SELECT
    first_name,
    last_name,
    salary,
    hire_date,
    AVG(salary) OVER (
        ORDER BY hire_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) as moving_avg_3_periods
FROM employees
ORDER BY hire_date;

-- Department-wise aggregations
SELECT
    first_name,
    last_name,
    department_id,
    salary,
    SUM(salary) OVER (PARTITION BY department_id) as dept_total_salary,
    COUNT(*) OVER (PARTITION BY department_id) as dept_size,
    salary::float / SUM(salary) OVER (PARTITION BY department_id) * 100 as salary_percentage_of_dept
FROM employees;
```

LAG and LEAD Functions

```sql
-- Compare with previous/next records
SELECT
    first_name,
    last_name,
    salary,
    hire_date,
    LAG(salary, 1) OVER (ORDER BY hire_date) as previous_hire_salary,
    LEAD(salary, 1) OVER (ORDER BY hire_date) as next_hire_salary,
    salary - LAG(salary, 1) OVER (ORDER BY hire_date) as salary_change,
    -- Multiple periods
    LAG(salary, 2) OVER (ORDER BY hire_date) as salary_2_hires_ago
FROM employees
ORDER BY hire_date;

-- Department-wise comparisons
SELECT
    first_name,
    last_name,
    department_id,
    salary,
    LAG(salary) OVER (PARTITION BY department_id ORDER BY salary DESC) as next_higher_salary_in_dept,
    LEAD(salary) OVER (PARTITION BY department_id ORDER BY salary DESC) as next_lower_salary_in_dept
FROM employees
ORDER BY department_id, salary DESC;
```

FIRST_VALUE and LAST_VALUE

```sql
-- Get first and last values in window
SELECT
    first_name,
    last_name,
    department_id,
    salary,
    FIRST_VALUE(salary) OVER (PARTITION BY department_id ORDER BY salary DESC) as highest_dept_salary,
    LAST_VALUE(salary) OVER (
        PARTITION BY department_id
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) as lowest_dept_salary
FROM employees;

-- Compare with department extremes
SELECT
    first_name,
    last_name,
    department_id,
    salary,
    FIRST_VALUE(first_name || ' ' || last_name) OVER (
        PARTITION BY department_id ORDER BY salary DESC
    ) as highest_earner_in_dept,
    salary - FIRST_VALUE(salary) OVER (
        PARTITION BY department_id ORDER BY salary DESC
    ) as difference_from_highest
FROM employees;
```

Advanced Window Frame Specifications

```
-- Different frame specifications
SELECT
    first_name,
    hire_date,
    salary,
    -- Default frame: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    SUM(salary) OVER (ORDER BY hire_date) as default_sum,
    -- Rows frame: exactly N rows
    SUM(salary) OVER (
        ORDER BY hire_date
        ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING
    ) as sum_5_rows_window,
    -- Range frame: based on value ranges
    SUM(salary) OVER (
        ORDER BY salary
        RANGE BETWEEN 5000 PRECEDING AND 5000 FOLLOWING
    ) as sum_salary_range_10k,
    -- Groups frame: based on groups of equal values
    SUM(salary) OVER (
        ORDER BY department_id
        GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) as sum_adjacent_groups
FROM employees;
```

---

13. Triggers & Functions

Creating Functions

```
-- Basic function
CREATE OR REPLACE FUNCTION calculate_annual_salary(monthly_salary DECIMAL)
RETURNS DECIMAL AS $
BEGIN
    RETURN monthly_salary * 12;
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT
    first_name,
    last_name,
    salary,
    calculate_annual_salary(salary) as annual_salary
FROM employees;

-- Function with multiple parameters and logic
CREATE OR REPLACE FUNCTION get_salary_grade(emp_salary DECIMAL, dept_id INTEGER)
RETURNS TEXT AS $
DECLARE
    avg_dept_salary DECIMAL;
    grade TEXT;
BEGIN
    -- Get average salary for the department
    SELECT AVG(salary) INTO avg_dept_salary
    FROM employees
    WHERE department_id = dept_id;

    -- Determine grade
    IF emp_salary >= avg_dept_salary * 1.2 THEN
        grade := 'A';
    ELSIF emp_salary >= avg_dept_salary THEN
        grade := 'B';
    ELSIF emp_salary >= avg_dept_salary * 0.8 THEN
        grade := 'C';
    ELSE
        grade := 'D';
    END IF;

    RETURN grade;
END;
$ LANGUAGE plpgsql;

-- Function returning table
CREATE OR REPLACE FUNCTION get_department_summary(dept_id INTEGER)
RETURNS TABLE(
    employee_count INTEGER,
    avg_salary DECIMAL,
    total_salary DECIMAL,
    min_salary DECIMAL,
    max_salary DECIMAL
) AS $
BEGIN
    RETURN QUERY
    SELECT
        COUNT(*)::INTEGER,
        AVG(salary)::DECIMAL,
```

```
        SUM(salary)::DECIMAL,
        MIN(salary)::DECIMAL,
        MAX(salary)::DECIMAL
    FROM employees e
    WHERE e.department_id = dept_id;
END;
$ LANGUAGE plpgsql;

-- Usage
SELECT * FROM get_department_summary(1);
```

Trigger Functions

```
-- Create audit log table
CREATE TABLE audit_log (
    id SERIAL PRIMARY KEY,
    table_name VARCHAR(50),
    operation VARCHAR(10),
    old_data JSONB,
    new_data JSONB,
    changed_by VARCHAR(100),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Audit trigger function
CREATE OR REPLACE FUNCTION audit_trigger_function()
RETURNS TRIGGER AS $
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (table_name, operation, old_data, changed_by)
        VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD), current_user);
        RETURN OLD;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log (table_name, operation, old_data, new_data, changed_by)
        VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD), row_to_json(NEW), current_user);
        RETURN NEW;
    ELSIF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (table_name, operation, new_data, changed_by)
        VALUES (TG_TABLE_NAME, TG_OP, row_to_json(NEW), current_user);
        RETURN NEW;
    END IF;
    RETURN NULL;
END;
$ LANGUAGE plpgsql;

-- Auto-update timestamp trigger
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$ LANGUAGE plpgsql;
```

Creating Triggers

```
-- Audit trigger on employees table
CREATE TRIGGER employees_audit_trigger
    AFTER INSERT OR UPDATE OR DELETE ON employees
    FOR EACH ROW
    EXECUTE FUNCTION audit_trigger_function();

-- Auto-update timestamp trigger
CREATE TRIGGER employees_update_timestamp
    BEFORE UPDATE ON employees
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();

-- Validation trigger
CREATE OR REPLACE FUNCTION validate_salary_increase()
RETURNS TRIGGER AS $
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.salary > OLD.salary * 1.5 THEN
        RAISE EXCEPTION 'Salary increase cannot exceed 50%% in one update. Old: %, New: %', OLD.salary, NEW.salary;
    END IF;
    RETURN NEW;
END;
$ LANGUAGE plpgsql;

CREATE TRIGGER salary_validation_trigger
    BEFORE UPDATE ON employees
    FOR EACH ROW
    EXECUTE FUNCTION validate_salary_increase();

-- Conditional trigger (PostgreSQL 9.0+)
CREATE TRIGGER high_salary_audit_trigger
    AFTER UPDATE ON employees
    FOR EACH ROW
    WHEN (NEW.salary > 100000)
    EXECUTE FUNCTION audit_trigger_function();
```

Advanced Function Examples

```
-- Function with error handling
CREATE OR REPLACE FUNCTION safe_divide(numerator DECIMAL, denominator DECIMAL)
RETURNS DECIMAL AS $
BEGIN
    IF denominator = 0 THEN
        RAISE NOTICE 'Division by zero attempted';
        RETURN NULL;
    END IF;
    RETURN numerator / denominator;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error in safe_divide: %', SQLERRM;
        RETURN NULL;
END;
$ LANGUAGE plpgsql;

-- Function with loops and arrays
CREATE OR REPLACE FUNCTION calculate_fibonacci(n INTEGER)
RETURNS INTEGER[] AS $
DECLARE
    result INTEGER[] := ARRAY[0, 1];
    i INTEGER;
BEGIN
    IF n <= 0 THEN
        RETURN ARRAY[]::INTEGER[];
    ELSIF n = 1 THEN
        RETURN ARRAY[0];
    ELSIF n = 2 THEN
        RETURN result;
    END IF;

    FOR i IN 3..n LOOP
        result := array_append(result, result[i-1] + result[i-2]);
    END LOOP;

    RETURN result;
END;
$ LANGUAGE plpgsql;

-- Recursive function
CREATE OR REPLACE FUNCTION factorial(n INTEGER)
RETURNS BIGINT AS $
BEGIN
    IF n <= 1 THEN
        RETURN 1;
    ELSE
        RETURN n * factorial(n - 1);
    END IF;
END;
$ LANGUAGE plpgsql;
```

14. Database Connectors & Change Data Capture

Logical Replication Setup

```
-- Enable logical replication (requires superuser)
-- In postgresql.conf:
-- wal_level = logical
-- max_replication_slots = 4
-- max_wal_senders = 4

-- Create publication for specific tables
CREATE PUBLICATION employees_pub FOR TABLE employees, departments;

-- Create publication for all tables
CREATE PUBLICATION all_tables_pub FOR ALL TABLES;

-- Create replication slot
SELECT pg_create_logical_replication_slot('my_slot', 'pgoutput');

-- Monitor replication slots
SELECT slot_name, plugin, slot_type, database, active, confirmed_flush_lsn
FROM pg_replication_slots;
```

Using pg_logical for Change Data Capture

```
-- Create a function to decode logical replication changes
CREATE OR REPLACE FUNCTION process_wal_changes()
RETURNS TABLE(lsn pg_lsn, xid xid, data text) AS $
BEGIN
    RETURN QUERY
    SELECT
        w.lsn,
        w.xid,
        w.data
    FROM pg_logical_slot_get_changes('my_slot', NULL, NULL) w;
END;
$ LANGUAGE plpgsql;

-- Example of consuming changes
SELECT * FROM process_wal_changes();
```

LISTEN/NOTIFY for Real-time Updates

```
-- Create notification function
CREATE OR REPLACE FUNCTION notify_employee_change()
RETURNS TRIGGER AS $
DECLARE
    notification JSON;
BEGIN
    notification = json_build_object(
        'operation', TG_OP,
        'table', TG_TABLE_NAME,
        'timestamp', CURRENT_TIMESTAMP
    );

    IF TG_OP = 'DELETE' THEN
        notification = notification || json_build_object('old_data', row_to_json(OLD));
        PERFORM pg_notify('employee_changes', notification::text);
        RETURN OLD;
    ELSE
        notification = notification || json_build_object('new_data', row_to_json(NEW));
        IF TG_OP = 'UPDATE' THEN
            notification = notification || json_build_object('old_data', row_to_json(OLD));
        END IF;
        PERFORM pg_notify('employee_changes', notification::text);
        RETURN NEW;
    END IF;
END;
$ LANGUAGE plpgsql;

-- Create trigger for notifications
CREATE TRIGGER employee_notify_trigger
    AFTER INSERT OR UPDATE OR DELETE ON employees
    FOR EACH ROW
    EXECUTE FUNCTION notify_employee_change();

-- Listen for notifications (in a client session)
LISTEN employee_changes;

-- Send custom notifications
NOTIFY employee_changes, '{"message": "Manual notification", "timestamp": "2023-01-01T12:00:00"}';
```

Event Sourcing Pattern

```
-- Create events table
CREATE TABLE employee_events (
    id SERIAL PRIMARY KEY,
    aggregate_id INTEGER NOT NULL, -- employee_id
    event_type VARCHAR(50) NOT NULL,
    event_data JSONB NOT NULL,
    event_version INTEGER NOT NULL,
    occurred_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(aggregate_id, event_version)
);

-- Function to append events
CREATE OR REPLACE FUNCTION append_employee_event(
    emp_id INTEGER,
    evt_type VARCHAR,
    evt_data JSONB
)
RETURNS INTEGER AS $
DECLARE
    next_version INTEGER;
    event_id INTEGER;
BEGIN
    -- Get next version for this aggregate
    SELECT COALESCE(MAX(event_version), 0) + 1
    INTO next_version
    FROM employee_events
    WHERE aggregate_id = emp_id;

    -- Insert event
    INSERT INTO employee_events (aggregate_id, event_type, event_data, event_version)
    VALUES (emp_id, evt_type, evt_data, next_version)
    RETURNING id INTO event_id;

    RETURN event_id;
END;
$ LANGUAGE plpgsql;

-- Function to rebuild aggregate from events
CREATE OR REPLACE FUNCTION get_employee_state(emp_id INTEGER)
RETURNS JSONB AS $
DECLARE
    current_state JSONB := '{}';
    evt RECORD;
BEGIN
    FOR evt IN
        SELECT event_type, event_data
        FROM employee_events
        WHERE aggregate_id = emp_id
        ORDER BY event_version
    LOOP
        CASE evt.event_type
            WHEN 'EmployeeCreated' THEN
                current_state := evt.event_data;
            WHEN 'SalaryChanged' THEN
                current_state := jsonb_set(current_state, '{salary}', evt.event_data->'new_salary');
            WHEN 'DepartmentChanged' THEN
                current_state := jsonb_set(current_state, '{department_id}', evt.event_data->'new_department_id');
```

```
        END CASE;
    END LOOP;

    RETURN current_state;
END;
$ LANGUAGE plpgsql;

-- Usage examples
SELECT append_employee_event(
    1,
    'EmployeeCreated',
    '{"first_name": "John", "last_name": "Doe", "salary": 75000, "department_id": 1}'
);

SELECT append_employee_event(
    1,
    'SalaryChanged',
    '{"old_salary": 75000, "new_salary": 80000, "reason": "Performance increase"}'
);
```

---

15. Audit Logging

Comprehensive Audit System

```
-- Enhanced audit log table
CREATE TABLE audit_logs (
    id SERIAL PRIMARY KEY,
    schema_name VARCHAR(100),
    table_name VARCHAR(100),
    operation VARCHAR(20),
    record_id INTEGER,
    old_values JSONB,
    new_values JSONB,
    changed_fields TEXT[],
    user_name VARCHAR(100),
    application_name VARCHAR(100),
    client_ip INET,
    session_id VARCHAR(100),
    transaction_id BIGINT,
    timestamp TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    statement TEXT
);

-- Create indexes for performance
CREATE INDEX idx_audit_logs_table_operation ON audit_logs(table_name, operation);
CREATE INDEX idx_audit_logs_timestamp ON audit_logs(timestamp);
CREATE INDEX idx_audit_logs_user ON audit_logs(user_name);
CREATE INDEX idx_audit_logs_record_id ON audit_logs(record_id);

-- Generic audit trigger function
CREATE OR REPLACE FUNCTION generic_audit_trigger()
RETURNS TRIGGER AS $
DECLARE
    old_values JSONB;
    new_values JSONB;
    changed_fields TEXT[] := ARRAY[]::TEXT[];
    record_id INTEGER;
BEGIN
    -- Get record ID (assuming 'id' column exists)
    IF TG_OP = 'DELETE' THEN
        record_id := OLD.id;
        old_values := to_jsonb(OLD);
        new_values := NULL;
    ELSIF TG_OP = 'INSERT' THEN
        record_id := NEW.id;
        old_values := NULL;
        new_values := to_jsonb(NEW);
    ELSIF TG_OP = 'UPDATE' THEN
        record_id := NEW.id;
        old_values := to_jsonb(OLD);
        new_values := to_jsonb(NEW);

        -- Identify changed fields
        SELECT ARRAY(
            SELECT key
            FROM jsonb_each(old_values) o
            WHERE NOT (new_values @> jsonb_build_object(o.key, o.value))
        ) INTO changed_fields;
    END IF;

    -- Insert audit record
```

```
    INSERT INTO audit_logs (
        schema_name,
        table_name,
        operation,
        record_id,
        old_values,
        new_values,
        changed_fields,
        user_name,
        application_name,
        client_ip,
        session_id,
        transaction_id,
        statement
    ) VALUES (
        TG_TABLE_SCHEMA,
        TG_TABLE_NAME,
        TG_OP,
        record_id,
        old_values,
        new_values,
        changed_fields,
        session_user,
        current_setting('application_name', true),
        inet_client_addr(),
        current_setting('log_line_prefix', true),
        txid_current(),
        current_query()
    );

    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$ LANGUAGE plpgsql;

-- Apply audit trigger to tables
CREATE TRIGGER employees_audit
    AFTER INSERT OR UPDATE OR DELETE ON employees
    FOR EACH ROW EXECUTE FUNCTION generic_audit_trigger();

CREATE TRIGGER departments_audit
    AFTER INSERT OR UPDATE OR DELETE ON departments
    FOR EACH ROW EXECUTE FUNCTION generic_audit_trigger();
```

Field-Level Auditing

```
-- Table for field-level audit trail
CREATE TABLE field_audit_trail (
    id SERIAL PRIMARY KEY,
    table_name VARCHAR(100),
    record_id INTEGER,
    field_name VARCHAR(100),
    old_value TEXT,
    new_value TEXT,
    changed_by VARCHAR(100),
    changed_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    change_reason TEXT
);

-- Field-level audit function
CREATE OR REPLACE FUNCTION field_level_audit_trigger()
RETURNS TRIGGER AS $
DECLARE
    old_record JSONB;
    new_record JSONB;
    field_name TEXT;
    old_value TEXT;
    new_value TEXT;
BEGIN
    IF TG_OP = 'UPDATE' THEN
        old_record := to_jsonb(OLD);
        new_record := to_jsonb(NEW);

        -- Loop through each field
        FOR field_name IN SELECT jsonb_object_keys(new_record) LOOP
            old_value := old_record ->> field_name;
            new_value := new_record ->> field_name;

            -- Only log if value changed
            IF old_value IS DISTINCT FROM new_value THEN
                INSERT INTO field_audit_trail (
                    table_name,
                    record_id,
                    field_name,
                    old_value,
                    new_value,
                    changed_by
                ) VALUES (
                    TG_TABLE_NAME,
                    NEW.id,
                    field_name,
                    old_value,
                    new_value,
                    session_user
                );
            END IF;
        END LOOP;
    END IF;

    RETURN NEW;
END;
$ LANGUAGE plpgsql;
```

Audit Queries and Reporting

```sql
-- View recent changes
SELECT
    timestamp,
    user_name,
    table_name,
    operation,
    record_id,
    changed_fields
FROM audit_logs
ORDER BY timestamp DESC
LIMIT 100;

-- Track changes to specific record
SELECT
    timestamp,
    operation,
    user_name,
    old_values,
    new_values,
    changed_fields
FROM audit_logs
WHERE table_name = 'employees' AND record_id = 1
ORDER BY timestamp;

-- User activity report
SELECT
    user_name,
    COUNT(*) as total_changes,
    COUNT(*) FILTER (WHERE operation = 'INSERT') as inserts,
    COUNT(*) FILTER (WHERE operation = 'UPDATE') as updates,
    COUNT(*) FILTER (WHERE operation = 'DELETE') as deletes,
    MIN(timestamp) as first_activity,
    MAX(timestamp) as last_activity
FROM audit_logs
WHERE timestamp >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY user_name
ORDER BY total_changes DESC;

-- Most frequently changed tables
SELECT
    table_name,
    COUNT(*) as change_count,
    COUNT(DISTINCT record_id) as unique_records_changed,
    COUNT(DISTINCT user_name) as users_making_changes
FROM audit_logs
WHERE timestamp >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY table_name
ORDER BY change_count DESC;

-- Audit trail for compliance
CREATE VIEW compliance_audit_trail AS
SELECT
    id,
    timestamp as event_time,
    user_name as who,
    table_name || '.' || COALESCE(record_id::TEXT, 'unknown') as what,
    operation as action,
```

```
    client_ip as from_where,
    CASE
        WHEN operation = 'INSERT' THEN 'Created record'
        WHEN operation = 'UPDATE' THEN 'Modified fields: ' || array_to_string(changed_fields, ', ')
        WHEN operation = 'DELETE' THEN 'Deleted record'
    END as description
FROM audit_logs
ORDER BY timestamp DESC;
```

Audit Data Retention

```
-- Function to archive old audit logs
CREATE OR REPLACE FUNCTION archive_old_audit_logs(retention_days INTEGER DEFAULT 365)
RETURNS INTEGER AS $
DECLARE
    archived_count INTEGER;
BEGIN
    -- Move old records to archive table
    WITH archived AS (
        DELETE FROM audit_logs
        WHERE timestamp < CURRENT_DATE - INTERVAL '1 day' * retention_days
        RETURNING *
    )
    INSERT INTO audit_logs_archive SELECT * FROM archived;

    GET DIAGNOSTICS archived_count = ROW_COUNT;

    RAISE NOTICE 'Archived % audit log records older than % days', archived_count, retention_days;

    RETURN archived_count;
END;
$ LANGUAGE plpgsql;

-- Create archive table
CREATE TABLE audit_logs_archive (LIKE audit_logs INCLUDING ALL);

-- Schedule regular archiving (requires pg_cron extension)
-- SELECT cron.schedule('archive-audit-logs', '0 2 * * 0', 'SELECT archive_old_audit_logs(365);');
```

---

16. Performance & Optimization

Index Types and Usage

```sql
-- B-tree index (default, good for equality and range queries)
CREATE INDEX idx_employees_salary ON employees(salary);
CREATE INDEX idx_employees_name ON employees(last_name, first_name);

-- Partial index (only index rows meeting condition)
CREATE INDEX idx_active_employees ON employees(department_id)
WHERE status = 'active';

-- Expression index
CREATE INDEX idx_employees_full_name ON employees(LOWER(first_name || ' ' || last_name));

-- Hash index (only for equality)
CREATE INDEX idx_employees_email_hash ON employees USING HASH(email);

-- GIN index for full-text search
CREATE INDEX idx_employees_search ON employees USING GIN(to_tsvector('english', first_name || ' ' || last_name));

-- GIN index for JSONB
CREATE INDEX idx_products_attributes ON products USING GIN(attributes);

-- GiST index for geometric data
CREATE INDEX idx_locations_point ON locations USING GIST(coordinates);

-- Check index usage
SELECT
    schemaname,
    tablename,
    indexname,
    idx_tup_read,
    idx_tup_fetch,
    idx_scan
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC;

-- Find unused indexes
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) as size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Query Optimization

```
-- Use EXPLAIN to analyze query performance
EXPLAIN (ANALYZE, BUFFERS)
SELECT e.first_name, e.last_name, d.name
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.salary > 70000;

-- Optimize with proper indexing
CREATE INDEX idx_employees_salary_dept ON employees(salary, department_id);

-- Use covering indexes to avoid table lookups
CREATE INDEX idx_employees_covering ON employees(department_id, salary)
INCLUDE (first_name, last_name, email);

-- Optimize subqueries with EXISTS instead of IN when possible
-- Instead of:
SELECT * FROM departments
WHERE id IN (SELECT department_id FROM employees WHERE salary > 80000);

-- Use:
SELECT * FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.department_id = d.id AND e.salary > 80000);

-- Use LIMIT when you don't need all results
SELECT * FROM employees ORDER BY salary DESC LIMIT 10;

-- Use appropriate JOIN types
-- Inner join is faster than left join when you don't need unmatched rows
SELECT e.first_name, d.name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;
```

Table Partitioning

```
-- Range partitioning by date
CREATE TABLE sales (
    id SERIAL,
    sale_date DATE NOT NULL,
    amount DECIMAL(10,2),
    customer_id INTEGER
) PARTITION BY RANGE (sale_date);

-- Create partitions
CREATE TABLE sales_2023_q1 PARTITION OF sales
FOR VALUES FROM ('2023-01-01') TO ('2023-04-01');

CREATE TABLE sales_2023_q2 PARTITION OF sales
FOR VALUES FROM ('2023-04-01') TO ('2023-07-01');

CREATE TABLE sales_2023_q3 PARTITION OF sales
FOR VALUES FROM ('2023-07-01') TO ('2023-10-01');

CREATE TABLE sales_2023_q4 PARTITION OF sales
FOR VALUES FROM ('2023-10-01') TO ('2024-01-01');

-- Hash partitioning
CREATE TABLE user_data (
    id SERIAL,
    user_id INTEGER NOT NULL,
    data JSONB
) PARTITION BY HASH (user_id);

CREATE TABLE user_data_0 PARTITION OF user_data FOR VALUES WITH (modulus 4, remainder 0);
CREATE TABLE user_data_1 PARTITION OF user_data FOR VALUES WITH (modulus 4, remainder 1);
CREATE TABLE user_data_2 PARTITION OF user_data FOR VALUES WITH (modulus 4, remainder 2);
CREATE TABLE user_data_3 PARTITION OF user_data FOR VALUES WITH (modulus 4, remainder 3);

-- List partitioning
CREATE TABLE regional_sales (
    id SERIAL,
    region VARCHAR(10) NOT NULL,
    sale_amount DECIMAL(10,2)
) PARTITION BY LIST (region);

CREATE TABLE regional_sales_north PARTITION OF regional_sales FOR VALUES IN ('North', 'Northeast');
CREATE TABLE regional_sales_south PARTITION OF regional_sales FOR VALUES IN ('South', 'Southeast');
CREATE TABLE regional_sales_west PARTITION OF regional_sales FOR VALUES IN ('West', 'Southwest');
```

Performance Monitoring

```
-- Monitor slow queries
SELECT
    query,
    calls,
    total_time,
    mean_time,
    rows
FROM pg_stat_statements
ORDER BY mean_time DESC
LIMIT 10;

-- Check table sizes
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) as total_size,
    pg_size_pretty(pg_relation_size(schemaname||'.'||tablename)) as table_size,
    pg_size_pretty(pg_indexes_size(schemaname||'.'||tablename)) as indexes_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- Check database connections
SELECT
    datname as database,
    state,
    COUNT(*) as connection_count
FROM pg_stat_activity
GROUP BY datname, state
ORDER BY connection_count DESC;

-- Monitor buffer cache hit ratio
SELECT
    'buffer_cache_hit_ratio' as metric,
    ROUND(
        (sum(heap_blks_hit) / (sum(heap_blks_hit) + sum(heap_blks_read))) * 100, 2
    ) as percentage
FROM pg_statio_user_tables;

-- Check lock activity
SELECT
    l.mode,
    l.locktype,
    l.relation::regclass as table_name,
    l.pid,
    a.query
FROM pg_locks l
JOIN pg_stat_activity a ON l.pid = a.pid
WHERE l.granted = false;

-- Vacuum and analyze statistics
SELECT
    schemaname,
    tablename,
    last_vacuum,
    last_autovacuum,
    last_analyze,
```

```
    last_autoanalyze,
    n_tup_ins + n_tup_upd + n_tup_del as total_modifications
FROM pg_stat_user_tables
ORDER BY total_modifications DESC;
```

VACUUM and ANALYZE

```
-- Manual vacuum operations
VACUUM employees;                    -- Basic vacuum
VACUUM FULL employees;               -- Full vacuum (locks table)
VACUUM ANALYZE employees;            -- Vacuum and update statistics

-- Analyze for query planner statistics
ANALYZE employees;
ANALYZE;  -- Analyze entire database

-- Check vacuum progress
SELECT
    p.pid,
    p.datname,
    p.relid::regclass as table_name,
    p.phase,
    p.heap_blks_total,
    p.heap_blks_scanned,
    p.heap_blks_vacuumed
FROM pg_stat_progress_vacuum p;

-- Auto-vacuum configuration
SELECT name, setting, unit, short_desc
FROM pg_settings
WHERE name LIKE 'autovacuum%'
ORDER BY name;

-- Table bloat estimation
SELECT
    schemaname,
    tablename,
    ROUND(CASE WHEN otta=0 THEN 0.0 ELSE sml.relpages/otta::numeric END,1) AS tbloat,
    CASE WHEN relpages < otta THEN 0 ELSE bs*(sml.relpages-otta)::bigint END AS wastedbytes,
    pg_size_pretty((CASE WHEN relpages < otta THEN 0 ELSE bs*(sml.relpages-otta)::bigint END)::bigint) AS wastedsize
FROM (
    SELECT
        schemaname, tablename, cc.reltuples, cc.relpages, bs,
        CEIL((cc.reltuples*((datahdr+ma-
            (CASE WHEN datahdr%ma=0 THEN ma ELSE datahdr%ma END))+nullhdr2+4))/(bs-20::float)) AS otta
    FROM (
        SELECT
            ma,bs,schemaname,tablename,
            (datawidth+(hdr+ma-(case when hdr%ma=0 THEN ma ELSE hdr%ma END)))::numeric AS datahdr,
            (hdr+ma-(case when hdr%ma=0 THEN ma ELSE hdr%ma END)) AS nullhdr2
        FROM (
            SELECT
                schemaname, tablename, hdr, ma, bs,
                SUM((1-null_frac)*avg_width) AS datawidth,
                MAX(null_frac) AS maxfracsum
            FROM pg_stats s2
            JOIN (
                SELECT
                    23 AS hdr, 8 AS ma, 8192 AS bs
            ) AS constants ON true
            GROUP BY 1,2,3,4,5
        ) AS foo
    ) AS rs
    JOIN pg_class cc ON cc.relname = rs.tablename
```

```
    JOIN pg_namespace nn ON cc.relnamespace = nn.oid AND nn.nspname = rs.schemaname
) AS sml
ORDER BY wastedbytes DESC;
```

---

17. Security & User Management

User and Role Management

```
-- Create users
CREATE USER app_user WITH PASSWORD 'secure_password123';
CREATE USER readonly_user WITH PASSWORD 'readonly_pass456';
CREATE USER admin_user WITH PASSWORD 'admin_pass789' CREATEDB CREATEROLE;

-- Create roles
CREATE ROLE hr_role;
CREATE ROLE finance_role;
CREATE ROLE developer_role;

-- Grant roles to users
GRANT hr_role TO app_user;
GRANT readonly_user TO hr_role;

-- Role inheritance
CREATE ROLE base_user;
CREATE ROLE power_user INHERIT;  -- Will inherit base_user permissions
GRANT base_user TO power_user;

-- Set default role
ALTER USER app_user SET ROLE hr_role;

-- Password policies (requires passwordcheck extension)
-- ALTER SYSTEM SET shared_preload_libraries = 'passwordcheck';
-- ALTER USER app_user PASSWORD 'ComplexPassword123!';
```

Database and Schema Permissions

```
-- Database permissions
GRANT CONNECT ON DATABASE company_db TO app_user;
GRANT CREATE ON DATABASE company_db TO developer_role;

-- Schema permissions
GRANT USAGE ON SCHEMA public TO hr_role;
GRANT CREATE ON SCHEMA public TO developer_role;
GRANT ALL ON SCHEMA hr TO hr_role;

-- Table permissions
GRANT SELECT ON employees TO readonly_user;
GRANT SELECT, INSERT, UPDATE ON employees TO hr_role;
GRANT ALL ON employees TO admin_user;

-- Column-level permissions
GRANT SELECT (first_name, last_name, email) ON employees TO hr_role;
GRANT UPDATE (salary) ON employees TO finance_role;

-- Sequence permissions
GRANT USAGE ON SEQUENCE employees_id_seq TO hr_role;

-- Function permissions
GRANT EXECUTE ON FUNCTION calculate_annual_salary(DECIMAL) TO hr_role;
```

Row Level Security (RLS)

```
-- Enable RLS on table
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

-- Create policies
CREATE POLICY employee_self_policy ON employees
    FOR SELECT
    USING (email = current_user || '@company.com');

CREATE POLICY hr_department_policy ON employees
    FOR ALL
    TO hr_role
    USING (department_id IN (SELECT id FROM departments WHERE name IN ('HR', 'Administration')));

CREATE POLICY manager_policy ON employees
    FOR SELECT
    USING (manager_id = (SELECT id FROM employees WHERE email = current_user || '@company.com'));

-- Policy with function
CREATE OR REPLACE FUNCTION is_hr_user()
RETURNS BOOLEAN AS $
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM pg_auth_members m
        JOIN pg_roles r ON m.roleid = r.oid
        WHERE r.rolname = 'hr_role' AND m.member = (
            SELECT oid FROM pg_roles WHERE rolname = current_user
        )
    );
END;
$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE POLICY hr_access_policy ON employees
    FOR ALL
    USING (is_hr_user());

-- Bypass RLS for specific users
ALTER TABLE employees FORCE ROW LEVEL SECURITY;  -- Apply to table owners too
GRANT BYPASSRLS ON employees TO admin_user;
```

Security Best Practices

```
-- Create application-specific schemas
CREATE SCHEMA app;
CREATE SCHEMA audit;
CREATE SCHEMA config;

-- Limit public schema usage
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON DATABASE company_db FROM PUBLIC;

-- Create security barrier views
CREATE VIEW secure_employee_view WITH (security_barrier) AS
SELECT
    id,
    first_name,
    last_name,
    email,
    department_id,
    CASE
        WHEN current_user IN ('hr_user', 'admin_user') THEN salary
        ELSE NULL
    END as salary
FROM employees
WHERE
    current_user = 'admin_user'
    OR department_id = (
        SELECT department_id FROM employees
        WHERE email = current_user || '@company.com'
    );

-- Audit login attempts
CREATE TABLE login_attempts (
    id SERIAL PRIMARY KEY,
    username VARCHAR(100),
    ip_address INET,
    success BOOLEAN,
    attempted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Function to log login attempts (called from application)
CREATE OR REPLACE FUNCTION log_login_attempt(
    p_username VARCHAR,
    p_ip_address INET,
    p_success BOOLEAN
)
RETURNS VOID AS $
BEGIN
    INSERT INTO login_attempts (username, ip_address, success)
    VALUES (p_username, p_ip_address, p_success);

    -- Alert on multiple failed attempts
    IF NOT p_success THEN
        DECLARE
            recent_failures INTEGER;
        BEGIN
            SELECT COUNT(*) INTO recent_failures
            FROM login_attempts
            WHERE username = p_username
```

```
              AND success = false
              AND attempted_at >= NOW() - INTERVAL '15 minutes';

            IF recent_failures >= 5 THEN
                -- Could send notification or temporarily lock account
                RAISE NOTICE 'Multiple failed login attempts for user: %', p_username;
            END IF;
        END;
    END IF;
END;
$ LANGUAGE plpgsql SECURITY DEFINER;
```

SSL and Connection Security

```
-- Check SSL status
SELECT ssl, client_addr, application_name
FROM pg_stat_ssl
JOIN pg_stat_activity USING (pid);

-- Force SSL connections (in postgresql.conf)
-- ssl = on
-- ssl_cert_file = 'server.crt'
-- ssl_key_file = 'server.key'
-- ssl_ca_file = 'ca.crt'

-- pg_hba.conf entries for SSL
-- hostssl all all 0.0.0.0/0 cert
-- hostssl all all ::/0 cert

-- Check connection encryption
\conninfo

-- Monitor connections
SELECT
    usename,
    application_name,
    client_addr,
    state,
    query_start,
    CASE
        WHEN ssl THEN 'SSL'
        ELSE 'No SSL'
    END as encryption
FROM pg_stat_activity
WHERE state = 'active';
```

Data Encryption

```
-- Install pgcrypto extension
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Hash passwords
SELECT crypt('password123', gen_salt('bf'));

-- Encrypt sensitive data
CREATE TABLE secure_data (
    id SERIAL PRIMARY KEY,
    user_id INTEGER,
    encrypted_ssn BYTEA,  -- Store encrypted
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert encrypted data
INSERT INTO secure_data (user_id, encrypted_ssn)
VALUES (1, pgp_sym_encrypt('123-45-6789', 'encryption_key'));

-- Decrypt data
SELECT
    id,
    user_id,
    pgp_sym_decrypt(encrypted_ssn, 'encryption_key') as ssn
FROM secure_data;

-- Use different keys for different types of data
CREATE TABLE encryption_keys (
    key_name VARCHAR(50) PRIMARY KEY,
    key_value TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Function to get encryption key securely
CREATE OR REPLACE FUNCTION get_encryption_key(key_name TEXT)
RETURNS TEXT AS $
DECLARE
    key_value TEXT;
BEGIN
    SELECT encryption_keys.key_value INTO key_value
    FROM encryption_keys
    WHERE encryption_keys.key_name = get_encryption_key.key_name;

    IF key_value IS NULL THEN
        RAISE EXCEPTION 'Encryption key not found: %', key_name;
    END IF;

    RETURN key_value;
END;
$ LANGUAGE plpgsql SECURITY DEFINER;
```

Advanced Topics & Best Practices

Connection Pooling

```
-- Monitor connection usage
SELECT
    COUNT(*) as total_connections,
    COUNT(*) FILTER (WHERE state = 'active') as active_connections,
    COUNT(*) FILTER (WHERE state = 'idle') as idle_connections,
    COUNT(*) FILTER (WHERE state = 'idle in transaction') as idle_in_transaction
FROM pg_stat_activity;

-- Find long-running transactions
SELECT
    pid,
    usename,
    application_name,
    state,
    query_start,
    now() - query_start as duration,
    query
FROM pg_stat_activity
WHERE state != 'idle'
  AND query_start < now() - interval '5 minutes'
ORDER BY duration DESC;

-- Kill problematic connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND query_start < now() - interval '1 hour';
```

Backup and Recovery

```
-- Continuous archiving setup (in postgresql.conf)
-- archive_mode = on
-- archive_command = 'cp %p /backup/archive/%f'
-- wal_level = replica

-- Point-in-time recovery
-- pg_basebackup -D /backup/base -Ft -z -P
-- Create recovery.conf with: restore_command = 'cp /backup/archive/%f %p'

-- Logical backup with pg_dump
-- pg_dump -h localhost -U postgres -d company_db -f backup.sql
-- pg_dump -h localhost -U postgres -d company_db -Fc -f backup.custom

-- Restore from logical backup
-- psql -h localhost -U postgres -d company_db -f backup.sql
-- pg_restore -h localhost -U postgres -d company_db backup.custom
```

Performance Testing

```
-- Generate test data
INSERT INTO employees (first_name, last_name, email, salary, department_id)
SELECT
    'FirstName' || generate_series,
    'LastName' || generate_series,
    'user' || generate_series || '@company.com',
    50000 + (random() * 50000)::INTEGER,
    (random() * 5 + 1)::INTEGER
FROM generate_series(1, 100000);

-- Benchmark queries
\timing on
SELECT COUNT(*) FROM employees WHERE salary > 75000;
\timing off

-- Use pg_stat_statements for query analysis
SELECT
    substring(query, 1, 50) as short_query,
    calls,
    total_time,
    mean_time,
    stddev_time,
    rows
FROM pg_stat_statements
ORDER BY total_time DESC
LIMIT 10;
```

Conclusion

This comprehensive PostgreSQL guide covers:

1. **Foundation**: Basic setup, data types, and table management
2. **Core Operations**: CRUD operations, joins, filtering, and aggregation
3. **Advanced Querying**: JSON operations, subqueries, CTEs, and window functions
4. **Advanced Concepts**: Triggers, functions, change data capture, and audit logging
5. **Performance**: Indexing, optimization, partitioning, and monitoring
6. **Security**: User management, permissions, RLS, and encryption

Key Takeaways for Mastery:

1. **Practice Regularly**: Set up a test database and practice these concepts
2. **Understand Execution Plans**: Use EXPLAIN to understand query performance
3. **Monitor Performance**: Regularly check pg_stat_* views
4. **Security First**: Always implement proper authentication and authorization
5. **Stay Updated**: PostgreSQL releases new features regularly

Next Steps:

- Set up replication and high availability
- Learn about extensions like PostGIS for geographic data
- Explore foreign data wrappers (FDW)
- Study advanced indexing strategies
- Practice disaster recovery procedures

Remember: PostgreSQL mastery comes from understanding not just the syntax, but when and why to use each feature. Start with the basics and gradually work your way through the advanced concepts while building real-world projects.